# LAPORAN PRAKTIKUM
# PEMROGRAMAN MOBILE
# MODUL 3

## BUILD A SCROLLABLE LIST

**Oleh:**

**Noor Khalisa NIM. 2410817220012**

**PROGRAM STUDI TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK**
**UNIVERSITAS LAMBUNG MANGKURAT**
**APRIL 2026**

# LEMBAR PENGESAHAN
# LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
# MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Khalisa
NIM : 2410817220012

| | |
|---|---|
| Menyetujui,<br>Asisten Praktikum | Mengetahui,<br>Dosen Penanggung Jawab Praktikum |
| Muhammad Erza Raihan<br>NIM. 2310817210027 | Irham Maulani Abdul Gani, S.Kom., M.Kom.<br>NIP. 199710312025061009 |

# DAFTAR ISI

LEMBAR PENGESAHAN ..... 2
DAFTAR ISI ..... 3
DAFTAR GAMBAR ..... 4
DAFTAR TABEL ..... 5
SOAL 1 ..... 6
B. Source Code ..... 7
C. Output Program ..... 32
D. Pembahasan ..... 40
SOAL 2 ..... 63
A. Pembahasan ..... 63
SOAL 3 ..... 65
A. Pembahasan ..... 65
TAUTAN GIT ..... 70

# DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose Homescreen ..... 32
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose Carousel ..... 32
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose Scroll Vertical ..... 33
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Detail ..... 33
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Multi Language ..... 34
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Multi Language ..... 34
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose Homescreen Indonesia ..... 35
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Detail Indonesia ..... 35
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose Homescreen Landscape ..... 36
Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose Detail Landscape ..... 36
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML Homescreen ..... 36
Gambar 12. Screenshot Hasil Jawaban Soal 1 XML Carousel ..... 37
Gambar 13. Screenshot Hasil Jawaban Soal 1 XML Detail ..... 37
Gambar 14. Screenshot Hasil Jawaban Soal 1 XML Multi Language ..... 38
Gambar 15. Screenshot Hasil Jawaban Soal 1 XML Multi Language ..... 38
Gambar 16. Screenshot Hasil Jawaban Soal 1 XML Scroll ..... 39
Gambar 17. Screenshot Hasil Jawaban Soal 1 XML Detail ..... 39
Gambar 18. Screenshot Hasil Jawaban Soal 1 XML Detail Landscape ..... 40
Gambar 19. Screenshot Hasil Jawaban Soal 1 XML Homescreen Landscape ..... 40

# DAFTAR TABEL

Tabel 1. Source Song.kt (Compose) ..... 7
Tabel 2. Source Song.kt (Compose) ..... 7
Tabel 3. Source Code MainActivity.kt (Compose) ..... 9
Tabel 4. Source Code Song.kt (XML) ..... 16
Tabel 5. Source Code SongData.kt (XML) ..... 16
Tabel 6. Source Code strings.xml eng (XML) ..... 18
Tabel 7. Source Code strings.xml id (XML) ..... 19
Tabel 8. Source Code item_song.xml (XML) ..... 20
Tabel 9. Source Code fragment_home.xml (XML) ..... 22
Tabel 10. Source Code fragment_detail.xml (XML) ..... 23
Tabel 11. Source Code fragment_language.xml (XML) ..... 25
Tabel 12. Source Code SongAdapter.kt (XML) ..... 26
Tabel 13. Source Code HomeFragment.kt (XML) ..... 27
Tabel 14. Source Code DetailFragment.kt (XML) ..... 28
Tabel 15. Source Code LanguageFragment.kt (XML) ..... 30
Tabel 16. Source Code MainActivity.kt (XML) ..... 31

# SOAL 1

**Soal Praktikum:**

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
    a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
    b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian Bahasa
11. Gunakan Icon dari library ‘material’ (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah “Item Highlight” yang menggunakan Carousel atau LazyRow dengan ketentuan:
    - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

## B. Source Code

Tabel 1. Source Song.kt (Compose)

```
package com.example.modul3_compose_baru

import java.io.Serializable

data class Song(
    val id: Int,
    val title: String,
    val albumName: String,
    val year: String,
    val imageResId: Int,
    val externalLink: String,
    val descriptionResId: Int
) : Serializable
```

Tabel 2. Source Song.kt (Compose)

```
package com.example.modul3_compose_baru

object SongData {
    val songs = listOf(
        Song(
            id = 1,
            title = "Nothing's Gonna Hurt You Baby",
            albumName = "I.",
            year = "2012",
            imageResId = R.drawable.album_i,
            externalLink                                =
"https://open.spotify.com/track/3W7KHojYGgYaoX9ogKO9hU?s
i=bc234cdf897d49b3",
            descriptionResId = R.string.desc_i
        ),
        Song(
            id = 2,
            title = "Flash",
            albumName = "Cigarettes After Sex",
            year = "2017",
            imageResId = R.drawable.album_cas,
            externalLink                                =
"https://open.spotify.com/track/6eknHC7HjAKchl23x2A7dW?s
i=7ed322498ba447be",
            descriptionResId = R.string.desc_flash
        ),
        Song(
            id = 3,
            title = "Opera House",
```

```
            albumName = "Cigarettes After Sex",
            year = "2017",
            imageResId = R.drawable.album_cas,
            externalLink                                    = "https://open.spotify.com/track/2Ddfm2NQwTRsf7YVlt258S?si=f70b90493a034663",
            descriptionResId = R.string.desc_opera
        ),
        Song(
            id = 4,
            title = "Sweet",
            albumName = "Cigarettes After Sex",
            year = "2017",
            imageResId = R.drawable.album_cas,
            externalLink                                    = "https://open.spotify.com/track/2KhrPRV0V1FS2l4eQMJUWt?si=77d6282cd2044d80",
            descriptionResId = R.string.desc_sweet
            ),
        Song(
            id = 5,
            title = "Sesame Syrup",
            albumName = "Crush",
            year = "2018",
            imageResId = R.drawable.album_crush,
            externalLink                                    = "https://open.spotify.com/track/6GcZ4pdtZgbf2ptgHG31bq?si=953557105c374872",
            descriptionResId = R.string.desc_sesame
        ),
        Song(
            id = 6,
            title = "Neon Moon",
            albumName = "Neon Moon",
            year = "2018",
            imageResId = R.drawable.album_neon_moon,
            externalLink                                    = "https://open.spotify.com/track/5r7QVaDpYeWMcbxxrkqb6H?si=489aa904e83f47f1",
            descriptionResId = R.string.desc_neon
        ),
        Song(
            id = 7,
            title = "Cry",
            albumName = "Cry",
            year = "2019",
            imageResId = R.drawable.album_cry,
```

```
            externalLink                                    =
"https://open.spotify.com/track/7mDTvYD2ieE4Q28XFziMfJ?si=89dea18975c047e9",
            descriptionResId = R.string.desc_cry
        )
    )
}
```

Tabel 3. Source Code MainActivity.kt (Compose)

```
package com.example.modul3_compose_baru

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatDelegate
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import
androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.os.LocaleListCompat
import androidx.navigation.NavController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
```

```
import androidx.navigation.navArgument

val LavenderBlush = Color(0xFFFFE5EC)
val PinkChampagne = Color(0xFFFFC2D1)
val Watermelon = Color(0xFFFB6F92)

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                AppNavigation()
            }
        }
    }
}

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = "home") {
        composable("home") { HomeScreen(navController) }
        composable(
            route = "detail/{songId}",
            arguments = listOf(navArgument("songId") {
                type = NavType.IntType
            })
        ) { backStackEntry ->
            val songId = backStackEntry.arguments?.getInt("songId")
            DetailScreen(navController, songId)
        }

        composable("language") { LanguageScreen(navController) }
    }
}

@Composable
fun HomeScreen(navController: NavController) {
    val context = LocalContext.current

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(LavenderBlush)
    ) {
```

```
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = stringResource(id = R.string.app_name),
                color = Color.Black,
                fontSize = 22.sp,
                fontWeight = FontWeight.Bold
            )
            IconButton(onClick = {
                navController.navigate("language")
            }) {
                Icon(
                    painter = painterResource(android.R.drawable.ic_menu_sort_alphabetically),
                    contentDescription = "Change Language",
                    tint = Watermelon
                )
            }
        }

        LazyRow(contentPadding = PaddingValues(horizontal = 16.dp),
            horizontalArrangement = Arrangement.spacedBy(16.dp)) {
            items(SongData.songs) { song ->
                SongItemCard(song = song, navController = navController, context = context, Modifier.fillParentMaxWidth())
            }
        }

        Spacer(modifier = Modifier.height(16.dp))

        LazyColumn(contentPadding = PaddingValues(start = 16.dp, end = 16.dp, bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)) {
            items(SongData.songs) { song ->
```

```
                SongItemCard(song = song, navController = navController, context = context, Modifier.fillMaxWidth())
            }
        }
    }
}

@Composable
fun SongItemCard(song: Song, navController: NavController, context: android.content.Context, modifier: Modifier = Modifier) {
    Card(
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = PinkChampagne),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
        modifier = modifier
    ) {
        Row(modifier = Modifier.fillMaxWidth().padding(12.dp)) {
            Image(
                painter = painterResource(id = song.imageResId),
                contentDescription = null,
                contentScale = ContentScale.Crop,
                modifier = Modifier.size(110.dp).clip(RoundedCornerShape(16.dp))
            )
            Spacer(modifier = Modifier.width(12.dp))

            Column(modifier = Modifier.fillMaxWidth()) {
                Text(
                    song.title,
                    color = Color.Black,
                    fontWeight = FontWeight.Bold,
                    fontSize = 15.sp)
                Spacer(modifier = Modifier.height(4.dp))
                Text(text = "${stringResource(R.string.label_album)} ${song.albumName}", color = Color.Black, fontSize = 14.sp)

                Spacer(modifier = Modifier.height(8.dp))
                Row(horizontalArrangement = Arrangement.End, modifier = Modifier.fillMaxWidth()) {
                    Button(
                        onClick = {
```

```
                            val         intent         = Intent(Intent.ACTION_VIEW, Uri.parse(song.externalLink))
context.startActivity(intent)
                        },
                        colors                          = ButtonDefaults.buttonColors(containerColor = Color.White, contentColor = Watermelon),
                        contentPadding                  = PaddingValues(horizontal = 12.dp, vertical = 4.dp)
                    )                                   { Text(stringResource(R.string.btn_listen),   fontSize   = 12.sp) }

                    Spacer(modifier                     = Modifier.width(8.dp))

                    Button(
                        onClick            =            { navController.navigate("detail/${song.id}") },
                        colors                          = ButtonDefaults.buttonColors(containerColor = Watermelon, contentColor = Color.White),
                        contentPadding                  = PaddingValues(horizontal = 12.dp, vertical = 4.dp)
                    )                                   { Text(stringResource(R.string.btn_detail),   fontSize   = 12.sp) }
                }
            }
        }
    }
}


@Composable
fun DetailScreen(navController: NavController, songId: Int?) {
    val song = SongData.songs.find { it.id == songId }

    if (song != null) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .background(LavenderBlush)
                .verticalScroll(rememberScrollState())
                .padding(16.dp)
        ) {
            Box(
```

```
                modifier = Modifier
                    .fillMaxWidth(),
                    contentAlignment = Alignment.Center
            ) {
                Image(
                    painter   =   painterResource(id   =
song.imageResId),
                    contentDescription = null,
                    contentScale = ContentScale.Crop,
                    modifier = Modifier
                        .size(320.dp)
                        .clip(RoundedCornerShape(16.dp))
                )
            }

            Spacer(modifier = Modifier.height(16.dp))
            Text(
                song.title,
                color = Color.Black,
                fontWeight = FontWeight.Bold,
                fontSize = 28.sp
            )
            Text(
                "${stringResource(R.string.label_album)}
${song.albumName}",
                color = Color.Black,
                fontSize = 16.sp,
                modifier = Modifier.padding(top = 4.dp)
            )
            Text(
                "${stringResource(R.string.label_year)}
${song.year}",
                color = Color.Black,
                fontSize = 16.sp,
                modifier = Modifier.padding(top = 4.dp)
            )
            Text(
                text                                    =
stringResource(song.descriptionResId),
                color = Color.Black,
                fontSize = 15.sp,
                lineHeight = 22.sp,
                modifier = Modifier.padding(top = 24.dp)
            )

            Spacer(modifier = Modifier.height(32.dp))
            Button(
                onClick                =                {
navController.popBackStack()},
```

```
                colors                                      =
ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
                modifier = Modifier.fillMaxWidth()
            ) { Text(stringResource(R.string.btn_back)) }
        }
    }
}

@Composable
fun LanguageScreen(navController: NavController) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(LavenderBlush)
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment                                 =
Alignment.CenterHorizontally
    ) {
        Text(
            text                                            =
stringResource(R.string.setting_language),
            color = Color.Black,
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 24.dp)
        )
        Button(
            onClick = { setAppLocale("en") },
            colors                                          =
ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
            modifier = Modifier.fillMaxWidth()
        ) { Text("English")}

        Spacer(modifier = Modifier.height(16.dp))
        Button(
            onClick = { setAppLocale("id") },
            colors                                          =
ButtonDefaults.buttonColors(containerColor = Watermelon,
contentColor = Color.White),
            modifier = Modifier.fillMaxWidth()
        ) { Text("Indonesia")}
    }
}

fun setAppLocale(languageTag: String) {
```

```
    val                    localeList                    =
LocaleListCompat.forLanguageTags(languageTag)
    AppCompatDelegate.setApplicationLocales(localeList)
}
```

Tabel 4. Source Code Song.kt (XML)

```
package com.example.modul3_xml

import java.io.Serializable

data class Song(
    val id: Int,
    val title: String,
    val albumName: String,
    val year: String,
    val imageResId: Int,
    val externalLink: String,
    val descriptionResId: Int
) : Serializable
```

Tabel 5. Source Code SongData.kt (XML)

```
package com.example.modul3_xml

object SongData {
    val songs = listOf(
        Song(
            1,
            "Nothing's Gonna Hurt You Baby",
            "I",
            "2012",
            R.drawable.album_i,

"https://open.spotify.com/track/3W7KHojYGgYaoX9ogKO9hU?s
i=bc234cdf897d49b3",
            R.string.desc_i
        ),
        Song(
            2,
            "Flash",
            "Cigarettes After Sex",
            "2017",
            R.drawable.album_cas,

"https://open.spotify.com/track/6eknHC7HjAKchl23x2A7dW?s
i=7ed322498ba447be",
```

```
            R.string.desc_flash
            ),
        Song(
            3,
            "Opera House",
            "Cigarettes After Sex",
            "2017",
            R.drawable.album_cas,
"https://open.spotify.com/track/2Ddfm2NQwTRsf7YVlt258S?si=f70b90493a034663",
            R.string.desc_opera
        ),
        Song(
            4,
            "Sweet",
            "Cigarettes After Sex",
            "2017",
            R.drawable.album_cas,
"https://open.spotify.com/track/2KhrPRV0V1FS2l4eQMJUWt?si=77d6282cd2044d80",
            R.string.desc_sweet
            ),
        Song(
            5,
            "Sesame Syrup",
            "Crush",
            "2018",
            R.drawable.album_crush,
"https://open.spotify.com/track/6GcZ4pdtZgbf2ptgHG31bq?si=953557105c374872",
            R.string.desc_sesame
            ),
        Song(
            6,
            "Neon Moon",
            "Neon Moon",
            "2018",
            R.drawable.album_neon_moon,
"https://open.spotify.com/track/5r7QVaDpYeWMcbxxrkqb6H?si=489aa904e83f47f1",
            R.string.desc_neon
            ),
```

```
        Song(
            7,
            "Cry",
            "Cry",
            "2019",
            R.drawable.album_cry,
"https://open.spotify.com/track/7mDTvYD2ieE4Q28XFziMfJ?si=89dea18975c047e9",
            R.string.desc_cry
            )
    )
}
```

Tabel 6. Source Code strings.xml eng (XML)

```
<resources>
    <string name="app_name">CAS Music</string>
    <string name="label_album">Album:</string>
    <string name="label_year">Release Year:</string>
    <string name="btn_listen">Listen</string>
    <string name="btn_detail">Detail</string>
    <string name="btn_back">Back to Home</string>
    <string name="setting_language">Select Language</string>

    <string name="desc_i">One of the most popular songs by CAS, this track acts as a \'grown-up\' lullaby and a promise of deep protection between two lovers. Recorded as an experimental improvisation in a university stairway, it tenderly captures the sweetness of their shared moments and a vow to safeguard each other from whatever the future holds.</string>
    <string name="desc_flash">Following two lonely souls who meet on a rainy night, this song captures a fleeting yet powerful encounter. They find an instant connection and escape into a temporary bubble of happiness before reality inevitably pulls them back, leaving them with a beautiful memory of a glimpse of heaven amidst the chaos.</string>
    <string name="desc_opera">Driven by deep passion and admiration, this song expresses a love so profound that the singer is willing to build an entire opera house in a secluded place just to properly convey the magnitude of his feelings to the one he loves.</string>
    <string name="desc_sweet">This track beautifully describes two lovers who share a mutual connection of true love, highlighting the pure sweetness of knowing your
```

```
partner loves you deeply without any need for second-
guessing or overthinking.</string>
    <string name="desc_sesame">Serving as the second
single before their upcoming album, this ambient dream-
pop track depicts sensual musings about a current lover,
featuring the signature intimate and evocative lyrics
typical of the band.</string>
    <string name="desc_neon">A mesmerizing cover of the
1992 Brooks &amp; Dunn country track, this song
beautifully tells the heartbreaking story of a man who has
lost the love of his life and is now stranded in a state
of misery and pain, unable to move on.</string>
    <string name="desc_cry">Told from the perspective of
the less affectionate partner, this song delves into the
dynamics of a toxic relationship where an imbalance in
giving and receiving affection constantly leads to pain
and heartache.</string>
</resources>
```

Tabel 7. Source Code strings.xml id (XML)

```
<resources>
    <string name="app_name">Musik CAS</string>
    <string name="label_album">Album:</string>
    <string name="label_year">Tahun Rilis:</string>
    <string name="btn_listen">Dengar</string>
    <string name="btn_detail">Detail</string>
    <string name="btn_back">Kembali ke Beranda</string>
    <string name="setting_language">Pilih
Bahasa</string>

    <string name="desc_i">Salah satu lagu paling populer
dari CAS, lagu ini berfungsi sebagai pengantar tidur
bagi orang dewasa dan janji perlindungan mendalam antara
dua kekasih. Direkam sebagai improvisasi eksperimental
di tangga universitas, lagu ini dengan lembut menangkap
manisnya momen kebersamaan mereka dan sumpah untuk
saling menjaga dari apa pun yang terjadi di masa
depan.</string>
    <string name="desc_flash">Mengisahkan dua jiwa
kesepian yang bertemu di malam hujan, lagu ini menangkap
pertemuan singkat namun penuh kekuatan. Mereka menemukan
koneksi instan dan melarikan diri ke dalam gelembung
kebahagiaan sementara sebelum kenyataan akhirnya menarik
mereka kembali, meninggalkan kenangan indah tentang
sekilas surga di tengah kekacauan.</string>
    <string name="desc_opera">Didorong oleh gairah dan
kekaguman yang mendalam, lagu ini mengekspresikan cinta
yang begitu besar sehingga penyanyinya rela membangun
```

```
seluruh gedung opera di tempat terpencil hanya untuk
menyampaikan besarnya perasaannya kepada orang yang
dicintainya.</string>
    <string name="desc_sweet">Lagu ini dengan indah
menggambarkan dua kekasih yang berbagi koneksi cinta
sejati, menyoroti kemurnian manis saat mengetahui
pasanganmu sangat mencintaimu tanpa perlu ragu atau
terlalu banyak berpikir.</string>
    <string name="desc_sesame">Sebagai singel kedua
sebelum album mendatang mereka, lagu ambient dream-pop
ini menggambarkan renungan sensual tentang kekasih saat
ini, menampilkan lirik intim dan evokatif khas dari band
tersebut.</string>
    <string name="desc_neon">Sebuah cover memukau dari
lagu country Brooks &amp; Dunn tahun 1992, lagu ini
dengan indah menceritakan kisah memilukan tentang
seorang pria yang kehilangan cinta dalam hidupnya dan
kini terdampar dalam penderitaan dan rasa sakit, tak
mampu melangkah maju.</string>
    <string name="desc_cry">Diceritakan dari sudut
pandang pasangan yang kurang penyayang, lagu ini
menggali dinamika hubungan toksik di mana
ketidakseimbangan dalam memberi dan menerima kasih
sayang terus-menerus berujung pada rasa sakit dan patah
hati.</string>
</resources>
```

Tabel 8. Source Code item_song.xml (XML)

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView

xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    app:cardBackgroundColor="#FFC2D1">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="12dp">

```

```
<com.google.android.material.imageview.ShapeableImageVie
w
            android:id="@+id/img_song"
            android:layout_width="110dp"
            android:layout_height="110dp"
            android:scaleType="centerCrop"
app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
app:shapeAppearanceOverlay="@style/RoundedCornerImage"/>

        <TextView
            android:id="@+id/tv_title"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="12dp"
            android:textColor="@color/black"
            android:textSize="16sp"
            android:textStyle="bold"
app:layout_constraintEnd_toStartOf="@+id/tv_year"
app:layout_constraintStart_toEndOf="@+id/img_song"
            app:layout_constraintTop_toTopOf="parent"/>

        <TextView
            android:id="@+id/tv_year"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="@color/black"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="parent"/>
        <TextView
            android:id="@+id/tv_label_album"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:text="@string/label_album"
            android:textColor="@color/black"
app:layout_constraintStart_toStartOf="@+id/tv_title"
app:layout_constraintTop_toBottomOf="@+id/tv_title"/>

        <TextView
            android:id="@+id/tv_album_name"
            android:layout_width="0dp"
```

```
            android:layout_height="wrap_content"
            android:layout_marginStart="4dp"
            android:textColor="@color/black"
            app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toEndOf="@+id/tv_label_album"
app:layout_constraintTop_toTopOf="@+id/tv_label_album"/>

        <Button
            android:id="@+id/btn_link"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="8dp"
            android:text="@string/btn_listen"
            android:textColor="#FB6F92"
            android:backgroundTint="@color/white"
app:layout_constraintBottom_toBottomOf="@+id/btn_detail"
app:layout_constraintEnd_toStartOf="@+id/btn_detail"
app:layout_constraintTop_toTopOf="@+id/btn_detail"/>

        <Button
            android:id="@+id/btn_detail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/btn_detail"
            android:textColor="@color/white"
            android:backgroundTint="#FB6F92"
app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintTop_toBottomOf="@+id/tv_album_name"
/>
    </androidx.constraintlayout.widget.ConstraintLayout>

</com.google.android.material.card.MaterialCardView>
```

Tabel 9. Source Code fragment_home.xml (XML)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFE5EC">

    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/app_name"
            android:textSize="22sp"
            android:textStyle="bold"/>

        <ImageButton
            android:id="@+id/btn_language"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"

android:background="?attr/selectableItemBackgroundBorder
less"

android:src="@android:drawable/ic_menu_sort_alphabetical
ly"
            app:tint="#FB6F92"/>
    </RelativeLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rv_highlight"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingStart="8dp"
        android:paddingEnd="8dp"
        android:clipToPadding="false"/>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rv_songs"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="8dp"/>

</LinearLayout>
```

Tabel 10. Source Code fragment_detail.xml (XML)

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFE5EC">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">


<com.google.android.material.imageview.ShapeableImageVie
w
            android:id="@+id/img_detail_cover"
            android:layout_width="320dp"
            android:layout_height="320dp"
            android:layout_gravity="center"
            android:scaleType="centerCrop"

app:shapeAppearanceOverlay="@style/RoundedCornerImage"/>

        <TextView
            android:id="@+id/tv_detail_title"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:textSize="28sp"
            android:textStyle="bold"/>

        <TextView
            android:id="@+id/tv_detail_album"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@color/black"
            android:textSize="16sp"/>

        <TextView
            android:id="@+id/tv_detail_year"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
```

```
            android:textColor="@color/black"
            android:textSize="16sp"/>

        <TextView
            android:id="@+id/tv_detail_desc"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:textColor="@color/black"
            android:textSize="15sp"
            android:lineSpacingExtra="6dp"/>

        <Button
            android:id="@+id/btn_back"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="32dp"
            android:backgroundTint="#FB6F92"
            android:textColor="@color/white"
            android:text="@string/btn_back"/>
    </LinearLayout>
</androidx.core.widget.NestedScrollView>
```

Tabel 11. Source Code fragment_language.xml (XML)

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp"
    android:background="#FFE5EC">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/setting_language"
        android:textStyle="bold"
        android:textSize="24sp"
        android:layout_marginBottom="32dp"/>

    <Button
        android:id="@+id/btn_english"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:backgroundTint="#FB6F92"
        android:textColor="@android:color/white"
        android:text="English"/>

    <Button
        android:id="@+id/btn_indonesia"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:backgroundTint="#FB6F92"
        android:textColor="@android:color/white"
        android:text="Bahasa Indonesia"/>
</LinearLayout>
```

Tabel 12. Source Code SongAdapter.kt (XML)

```
package com.example.modul3_xml

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.example.modul3_xml.databinding.ItemSongBinding

class SongAdapter(
    private val list: List<Song>,
    private val onDetailClick: (Song) -> Unit,
    private val onLinkClick: (String) -> Unit
) : RecyclerView.Adapter<SongAdapter.ViewHolder>() {

    class ViewHolder(val binding: ItemSongBinding) : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val binding = ItemSongBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val song = list[position]
        holder.binding.apply {
            imgSong.setImageResource(song.imageResId)
            tvTitle.text = song.title

            tvAlbumName.text = song.albumName

            btnDetail.setOnClickListener {
```

```
                onDetailClick(song)
            }
            btnLink.setOnClickListener {
                onLinkClick(song.externalLink)
            }
        }
    }

    override fun getItemCount(): Int {
        return list.size
    }
}
```

Tabel 13. Source Code HomeFragment.kt (XML)

```
package com.example.modul3_xml

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.PagerSnapHelper
import com.example.modul3_xml.databinding.FragmentHomeBinding

class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val adapter = SongAdapter(SongData.songs,
```

```
            onDetailClick = { song ->
                val    bundle    =    Bundle().apply    { putSerializable("song", song) }
                findNavController().navigate(R.id.action_home_to_detail, bundle)
            },
            onLinkClick = { url ->
                val  intent  =  Intent(Intent.ACTION_VIEW, Uri.parse(url))
                startActivity(intent)
            }
        )

        binding.rvSongs.layoutManager                  = LinearLayoutManager(requireContext())
        binding.rvSongs.adapter = adapter

        binding.rvHighlight.layoutManager              = LinearLayoutManager(requireContext(), LinearLayoutManager.HORIZONTAL, false)
        binding.rvHighlight.adapter = adapter
        val snapHelper = PagerSnapHelper()
        snapHelper.attachToRecyclerView(binding.rvHighlight)

        binding.btnLanguage.setOnClickListener {
        findNavController().navigate(R.id.action_home_to_language)
        }
    }
}
```

Tabel 14. Source Code DetailFragment.kt (XML)

```
package com.example.modul3_xml

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.navigation.fragment.findNavController
import com.example.modul3_xml.databinding.FragmentDetailBinding
class DetailFragment : Fragment() {
    private var _binding: FragmentDetailBinding? = null
```

```
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding = FragmentDetailBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val song = arguments?.getSerializable("song") as? Song
        if (song != null) {
            binding.imgDetailCover.setImageResource(song.imageResId)
            binding.tvDetailTitle.text = song.title

            val labelAlbum = getString(R.string.label_album)
            val labelYear = getString(R.string.label_year)

            binding.tvDetailAlbum.text = "$labelAlbum ${song.albumName}"
            binding.tvDetailYear.text = "$labelYear ${song.year}"
            binding.tvDetailDesc.text = getString(song.descriptionResId)
        }

        binding.btnBack.setOnClickListener {
            findNavController().navigateUp()
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tabel 15. Source Code LanguageFragment.kt (XML)

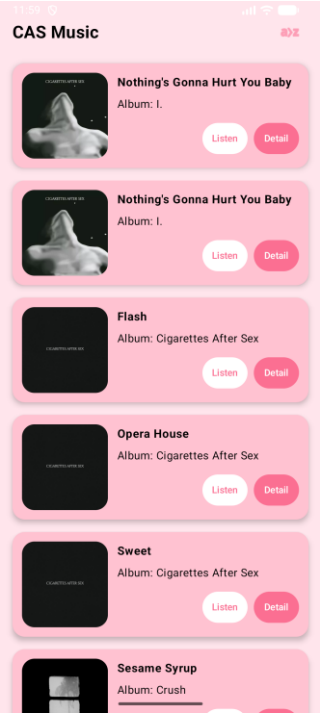
```
1   package com.example.modul3_xml
2
3   import android.os.Bundle
4   import android.view.LayoutInflater
5   import android.view.View
6   import android.view.ViewGroup
7   import androidx.appcompat.app.AppCompatDelegate
8   import androidx.core.os.LocaleListCompat
9   import androidx.fragment.app.Fragment
10  import
    com.example.modul3_xml.databinding.FragmentLanguageBindi
    ng
11  class LanguageFragment : Fragment() {
12      private var _binding: FragmentLanguageBinding? = null
13      private val binding get() = _binding!!
14
15      override fun onCreateView(
16          inflater: LayoutInflater,
17          container: ViewGroup?,
18          savedInstanceState: Bundle?
19      ): View? {
20          _binding                                        =
    FragmentLanguageBinding.inflate(inflater,      container,
    false)
21          return binding.root
22      }
23
24      override      fun      onViewCreated(view:      View,
    savedInstanceState: Bundle?) {
25          super.onViewCreated(view, savedInstanceState)
26
27          binding.btnEnglish.setOnClickListener {
28              setAppLocale("en")
29          }
30          binding.btnIndonesia.setOnClickListener {
31              setAppLocale("id")
32          }
33      }
34
35      private fun setAppLocale(languageTag: String) {
36          val                 localeList                  =
    LocaleListCompat.forLanguageTags(languageTag)
```

```
37
    AppCompatDelegate.setApplicationLocales(localeList)
38      }
39
40      override fun onDestroyView() {
41          super.onDestroyView()
42          _binding = null
43      }
44  }
```

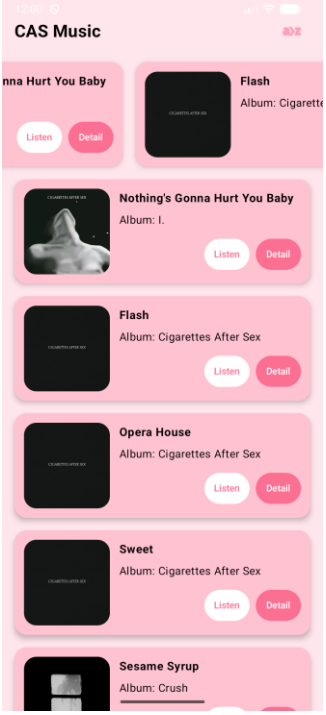
Tabel 16. Source Code MainActivity.kt (XML)

```
1   package com.example.modul3_xml
2
3   import android.os.Bundle
4   import androidx.appcompat.app.AppCompatActivity
5   import
    com.example.modul3_xml.databinding.ActivityMainBinding
6
7   class MainActivity : AppCompatActivity() {
8       private lateinit var binding: ActivityMainBinding
9
10      override fun onCreate(savedInstanceState: Bundle?) {
11          super.onCreate(savedInstanceState)
12          binding                                          =
    ActivityMainBinding.inflate(layoutInflater)
13          setContentView(binding.root)
14      }
15  }
```

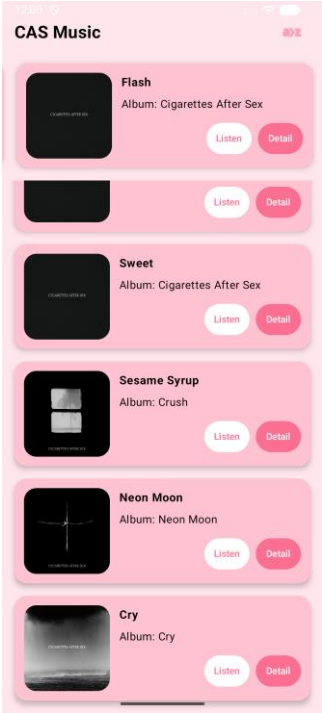
## C. Output Program



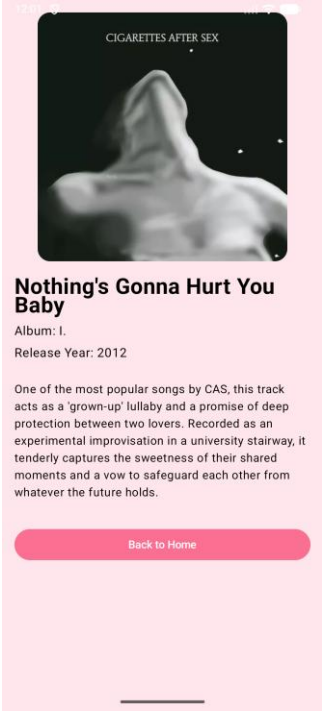

*Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose Homescreen*



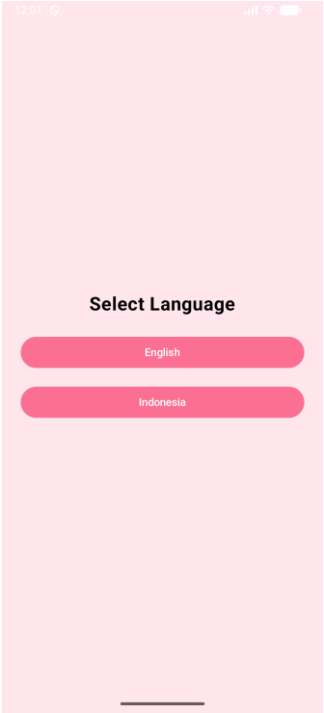

*Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose Carousel*




*Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose Scroll Vertical*

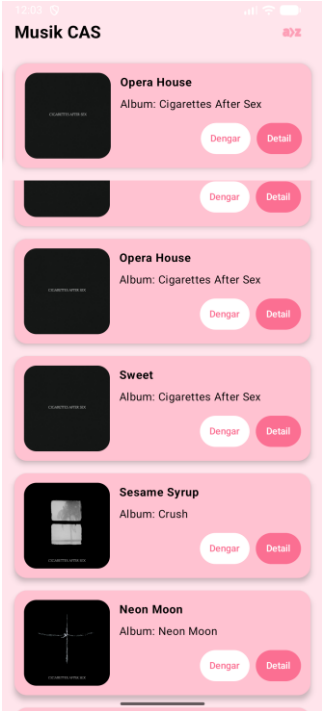



*Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose Detail*




*Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose Multi Language*

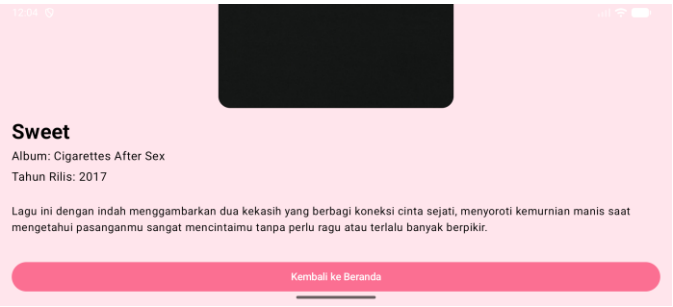
*Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose Multi Language*



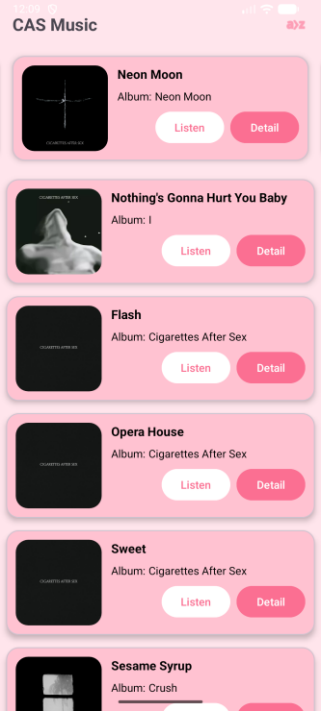

*Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose Homescreen Indonesia*

*Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Detail Indonesia*

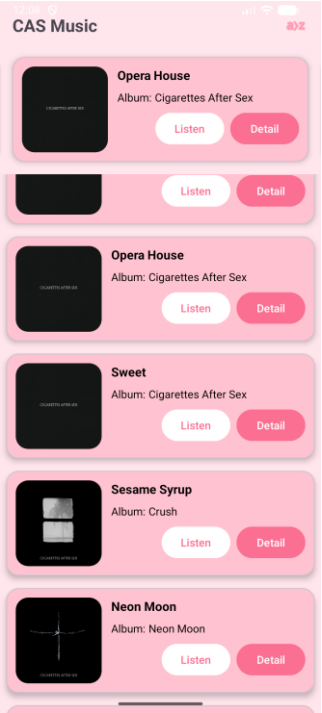
*Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose Homescreen Landscape*



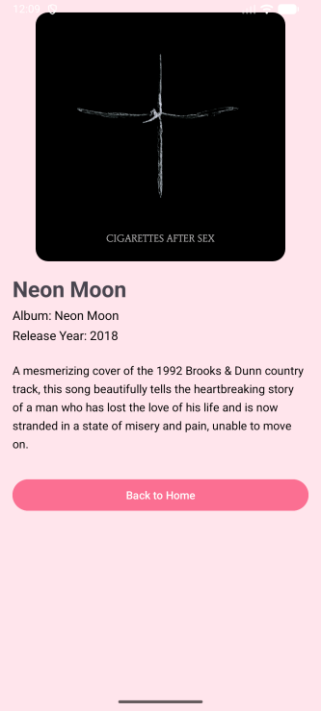

*Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose Detail Landscape*




*Gambar 11. Screenshot Hasil Jawaban Soal 1 XML Homescreen*



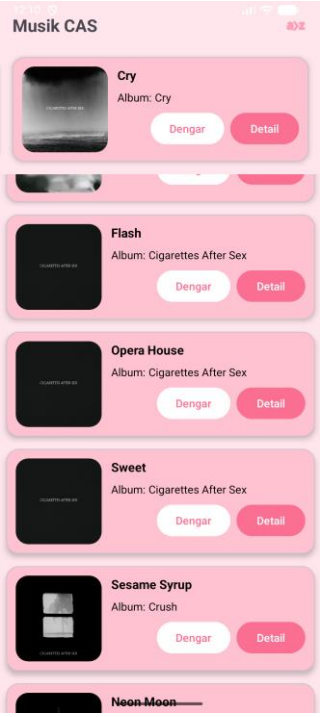

*Gambar 12. Screenshot Hasil Jawaban Soal 1 XML Carousel*




*Gambar 13. Screenshot Hasil Jawaban Soal 1 XML Detail*

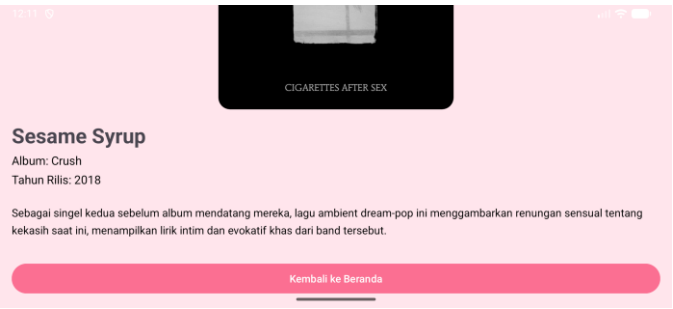
*Gambar 14. Screenshot Hasil Jawaban Soal 1 XML Multi Language*

*Gambar 15. Screenshot Hasil Jawaban Soal 1 XML Multi Language*

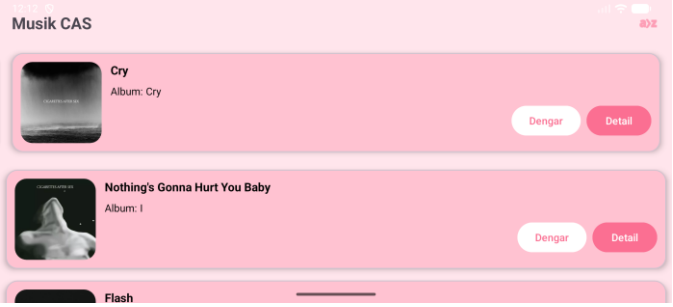



*Gambar 16. Screenshot Hasil Jawaban Soal 1 XML Scroll*

*Gambar 17. Screenshot Hasil Jawaban Soal 1 XML Detail*




*Gambar 18. Screenshot Hasil Jawaban Soal 1 XML Detail Landscape*




*Gambar 19. Screenshot Hasil Jawaban Soal 1 XML Homescreen Landscape*

## D. Pembahasan

### Song.Kt (Compose)

Pada baris [1] dan [3], deklarasi package menunjukkan lokasi direktori file, diikuti dengan pemanggilan pustaka bawaan Java yaitu java.io.Serializable.

Pada baris [5], dideklarasikan data class Song. Penggunaan kata kunci data menginstruksikan kompilator Kotlin untuk secara otomatis membuatkan kode berulang (boilerplate) tingkat utilitas seperti equals(), hashCode(), toString(), dan copy(). Hal ini menjadikan kelas ini murni berfokus sebagai wadah pembawa data.

Pada baris [6] sampai [12], properti title, albumName, year, dan externalLink menggunakan tipe data String. Sementara itu, id, imageResId, dan descriptionResId menggunakan tipe data Int. Penggunaan Int penting karena menyimpan ID referensi yang merujuk pada berkas fisik di dalam direktori res/drawable dan lokasi terjemahan di res/values/strings.xml

Pada baris [13], terdapat implementasi antarmuka : Serializable. Modifikasi inimemberikan kemampuan pada objek Song untuk dikonversi menjadi byte stream.

Secara arsitektural, ini memungkinkan objek lagu untuk untuk dikirim melintasi batas memori antar-komponen aplikasi, misalnya diteruskan melalui argumen sistem navigasi atau intent.

**SongData.Kt(Compose)**

Pada baris [3], object SongData. Berbeda dengan class biasa yang bisa diinstansiasi berulang kali, kata kunci object di Kotlin menerapkan design pattern Singleton. Ini menjamin bahwa di seluruh lifecycle aplikasi, memori perangkat hanya akan menyimpan satu buah salinan objek SongData ini saja. Pendekatan ini sangat efisien untuk menyediakan sumber data (*data source*) statis secara global.

Pada baris [4] sampai [69], variabel global val songs dideklarasikan yang diinisialisasi menggunakan fungsi listOf(...). Karena menggunakan listOf daripada mutableListOf, koleksi data yang dihasilkan bersifat *immutable* (tidak dapat dimodifikasi, ditambah, atau dihapus setelah aplikasi berjalan), memastikan data tetap aman dan konsisten saat dirender oleh LazyColumn atau LazyRow di lapisan UI.Di dalam blok listOf, dilakukan pemanggilan konstruktor Song(...) secara berulang untuk melakukan pemetaan *dummy data*. Setiap blok mengisi parameter sesuai kontrak data kelasnya secara berurutan, mengorganisasikan koleksi lagu.

**MainActivity.kt (Compose)**

Pada baris [1] sampai [39], package digunakan untuk mendeklarasikan direktori file program, sedangkan import digunakan untuk memanggil library dasar Android dan komponen Jetpack Compose. Pustaka yang dipanggil mencakup elemen UI dasar (Image, Text, Button), tata letak (Column, Row, LazyColumn, LazyRow), system navigasi (NavController, NavHost), serta library untuk fungsionalitas spesifik seperti Intent untuk membuka tautan eksternal dan AppCompatDelegate untuk manajemen bahasa aplikasi.

Pada baris [41] sampai [43], dilakukan inisialisasi variabel global berupa val LavenderBlush, PinkChampagne, dan Watermelon menggunakan class Color. Pendeklarasian ini bertujuan untuk membuat palet warna yang diinginkan, lalu ini

supaya warna antarmuka tetap konsistem dan memudahkan modifikasi tema di masa mendatang tanpa perlu mengubah kode hex satu per satu di setiap komponen.

Pada baris [45] sampai [54], class ‘MainActivity : AppCompatActivity()’ adalah deklarasi kelas utama. Terdapa perbedaan arsitektur, kelas ini mewarisi AppCompatActivity daripada ComponentActivity standar Compose. Hal ini diwajibkan karena aplikasi memanfaatkan fitur pergantian bahasa secara dinamis menggunakan AppCompatDelegate, yang hanya didukung oleh arsitektur backward-compatible dari kelas tersebut. Di dalam metode lifecycle override fun OnCreate(…), blok setContent {…} dipanggil untuk menggambar akar dari komponen UI, yaitu App Navigation.

Pada baris [56] sampai [74], @Composable fun AppNavigation berfungsi sebagai navigasi aplikasi. Dalam arsitektur Jetpack Compose, perpindahan antarlayar tidak lagi menggunakan Activity, melainkan menukar komponen Composable di dalam satu Activity Tunggal.

Pada baris [58], variabel navController diinisialisasi menggunakan rememberNavController(). Objek ini bertugas melacak posisi layar pengguna saat ini dan menyimpan riwayat tumpukan navigasi (back stack) agar pengguna bisa Kembali ke layar sebelumnya. remember memastikan status navigasi ini tidak hilang saat terjadi recomposition (render ulang UI).

Pada baris [60], komponen NavHost membungkus seluruh rute layar aplikasi. Atribut startDestination = ”home” secara eksplist memerintahkan aplikasi untuk menampilkan layar beranda pertama kali saat dibuka.

Pada baris [61], fungsi composable(”home”) mendaftarkan rute statis yang akan memanggil fungsi HomeScreen(navController).

Pada baris [62] sampai [70], ada implementasi dynamic routing. Rute didaftarkan sebagai “detail/{songId}”, yang berarti rute perlu sebuah parameter atau argument yang disisipkan pada URL internalnya. Antribut arguments = listOf(…) secara spesifik mendefinisikan bahwa argument “songId” yang ditangkap harus dikonversi dan

diperlukan sebagai data Integer (NavType.IntType). Variable backStackEntry kemudian mengekstrak argument songId tersebut, lalu meneruskannya ke dalam DetailScreen(navController, songId). Mekanisme ini memastikan layar detail merender lagu yang tepat sesuai dengan data yang ditekan pengguna di beranda.

Pada baris [72], rute statis “language” didaftarkan untuk mengakses menu pengaturan Bahasa.

Pada baris [76] sampai [125], @Composable fun HomeScreen() bertanggung jawab menyusun tata letak halaman utama aplikasi.

Pada baris [78], val context = LocalContext.current digunakan untuk mendapatkan context dari system operasi Android. Objek context ini wajib diteruskan ke SongItemCard karena aksi membuka link (implicit intent) membutuhkan context untuk menjalankan komponen di luar aplikasi.

Pada baris [80] sampai [84], wadah utama aplikasi dibungkus menggunakan Column dengan modifikasi fillMaxSize agar mengisi layar penuh, serta diberi warna latar belakang LavenderBlush.

Pada baris [85] sampai [107], ini adalah header. Ada Row dengan modifikasi fillMaxWidth() dan padding(16.dp). Atribut Arrangement.SpaceBetween sangat penting karena akan menolak elemen ke ujung kiri (teks judul) dan ujung kanan (tombol pengaturan). Komponen Text menampilkan nama aplikasi menggunakan stringResource, yang artinya teks ini mendukung multi language.

Pada baris [109] sampai [114], komponen LazyRow bertugas merender daftar lagu yang dapat digulir ke kiri dan kanan. PaddingValues(horizontal = 16.dp) memberikan jarak awal dan akhir agar elemen tidak menempel pada pinggir layar, Arrangement.spacedBy(16.dp) memberi spasi konsisten antarkartu. Lalu ada pemanggilan SongItemCard yang diberikan parameter Modifier.fillParentMaxWidth(). Modifikasi ini memaksa setiap kartu dalam daftar memiliki lebar sama persis dengan lebar layar perangkat, mengikuti dimensi parent vertical, sehingga ukurannya sama seperti kartu yang ada di daftar gulir atas-bawah.

Pada baris [118] sampai [123], komponen LazyColumn merender daftar lagu yang dapat digulir ke atas dan bawah. Dimodifikasi menggunakan Modifier.fillMaxWidth() secara standar, dan items(SongData.songs) melakukan perulangan dinamis untuk mencetak SongItemCard sebanyak jumlah lagu yang ada.

Pada baris [127] sampai [175], @Composable fun SongItemCard() adalah komponen antarmuka reuseable untuk mendesain satu buah kartu lagu.

Pada baris [129] sampai [134], fondasi menggunakan Card. Kartu ini diberi RoundedCornerShape(16.dp) agar keempat sudutnya melengkung, dengan warna khusus PinkChampagne, juga diberi efek bayangan menggunakan elevation sebesar 4.dp.

Pada baris [135], tata letak di dalam kartu dibelah menggunakan Row.

Pada baris [136] sampai [141], kartu diisi dengan komponen Image di sebelah kiri. Supaya gambar album proporsional dan tidak gepeng, digunakan property contentScale = ContentScale.Crop. Modifikasi clip(RoundedCornerShape(16.dp)) memastikan gambar terpotong melengkung mengikuti bentuk kartu di luarnya.

Pada baris [144] sampai [150], sebelah kanan kartu diisi dengan Column yang memuat teks judul dan album. Spacer digunakan untuk merenggankan jarak vertikal antar-teks.

Pada baris [154] sampai [171], bagian ini menyusun tombol Listen dan Detail dalam satu baris di ujung kanan kartu menggunakan Arrangement.End. Tombol Listen mengimplementasikan konsep implicit intent. Variabel intent menyimpan perintah Intent.ACTION_VIEW yang disertai konversi URL link lagu (Uri.parse(song.externalLink)). Saat tombol ditekan, context.startActivity(intent) akan diluncurkan, memaksa sistem operasi Android mencari aplikasi eksternal untuk membuka link tersebut. Kalau tombol Detail, tombol ini memicu navigasi internal aplikasi dengan mengeksekusi navController.navigate("detail/${song.id}). Perintah ini menyisipkan ID spesifik lagu yang sedang ditekan ke dalam rute, nanti akan ditangkap oleh AppNavigation().

Pada baris [178] sampai [240], @Composable fun DetailScreen() merender halaman informasi lengkap dari sebuah lagu yang dipilih.

Pada baris [180] dan [182], saat layar dimuat makan program langsung mencari data dari koleksi SongData.song menggunakan perintah .find { it.id == songId }. Ini memastikan antarmuka hanya akan menampilkan satu data objek tunggal yang ID-nya cocok dengan argumen yang diterima dari navigasi. Jika data ditemukan (if (song != null)), layar akan di-render.

Pada baris [183] sampai [189], konten utama dibungkus Column. Modifikasi verticalScroll(rememberScrollState()) diimplementasikan pada kolom ini, agar keseluruhan deskripsi layar yang panjang dapat digulir ke bawah dan tidak terpotong di ujung layar perangkat.

Pada baris [190] sampai [203], komponen Box bertindak sebagai wadah pembungkus dengan contentAlignment = Alignment.Center agar gambar yang ada di dalamnya presisi berada di tengah layar secara horizontal. Ini menampilkan cover album.

Pada baris [206] sampai [230], komponen Text disusun vertikal untuk menampilkan judul, nama, album, tahun rilis, dan deskripsi lengkap. Properti stringResource dikombinasikan dengan iterasi data Kotlin seperti "${stringResource(R.string.label_album)} ${song.albumName}" agar label bahasan mendukung pergantian bahasa, sementara isi datanya dinamis mengikuti lagu yang dipilih.

Pada baris [233] sampai [237], ujung bawah halaman ditutup dengan Button aksi kembali. Pada atribut onClick, perintah navController.popBackStack() dipanggil. Perintah ini berfungsi secara literal untuk keluar dari layar detail, keluar dari tumpukan memori navigasi, sehingga pengguna otomatis dikembalikan ke tumpukan layar sebelumnya.

Pada baris [242] sampai [272], @Composable fun LanguageScreen() merender antarmuka khusus untuk panel pengaturan lokalisasi Bahasa.

Pada baris [244] sampai [251], keseluruhan layar dikendalikan oleh satu Column dengan parameter pengaturan Arrangement.Center (berpusat di tengah vertical) dan Alignment.CenterHorizontally (berpusat di tengah horizontal). Kombinasi ini memastikan seluruh isi konten terkunci secara simetris di tengah-tengah layar.

Pada baris [259] sampai [270], terdapat dua tombol, yaitu tombol English dan Indonesia. Tombol-tombol ini tidak melakukan perpindahan layar secara navigasi, melainkan fungsi khusus setAppLocale("en") dan setAppLocale("id") sebagai event trigger pergantian lokalisasi aplikasi.

Pada baris [274] sampai [277], fun setAppLocale() adalah fungsi utilitas procedural Kotlin yang mengurus back-end pelokalan system Android modern.

Pada baris [], perintah LocaleListCompat.forLanguageTags(languageTag) menerima string kode bahasa (seperti "en" atau "id") dan mengonversinya ke dalam bentuk daftar local (locale).

Pada baris [276], perintah AppCompatDelegate.setApplicationLocales(localeList) kemudian menerapkan konfigurasi baru pada pada Application Context (tingkat aplikasi). Intruksi ini memaksa system untuk secara langsung mengambil nilai string dari file resource (strings.xml) yang berbeda dan memicu Compose untuk me-render ulang (recomposition) seluruh layar saat itu juga dengan bahasa yang baru, tanpa mengharuskan pengguna mematikan dan restart aplikasi.

**Song.kt (XML)**

Pada baris [1] dan [3], deklarasi package menunjukkan lokasi direktori file, diikuti dengan pemanggilan pustaka bawaan Java yaitu java.io.Serializable.

Pada baris [5], dideklarasikan data class Song. Penggunaan kata kunci data menginstruksikan kompilator Kotlin untuk secara otomatis membuatkan kode berulang (boilerplate) tingkat utilitas seperti equals(), hashCode(), toString(), dan copy(). Hal ini menjadikan kelas ini murni berfokus sebagai wadah pembawa data.

Pada baris [6] sampai [12], properti title, albumName, year, dan externalLink menggunakan tipe data String. Sementara itu, id, imageResId, dan descriptionResId menggunakan tipe data Int. Penggunaan Int penting karena menyimpan ID referensi yang merujuk pada berkas fisik di dalam direktori res/drawable dan lokasi terjemahan di res/values/strings.xml

Pada baris [13], terdapat implementasi antarmuka : Serializable. Modifikasi inimemberikan kemampuan pada objek Song untuk dikonversi menjadi byte stream. Secara arsitektural, ini memungkinkan objek lagu untuk untuk dikirim melintasi batas memori antar-komponen aplikasi, misalnya diteruskan melalui argumen sistem navigasi atau intent.

**SongData.kt (XML)**

Pada baris [3], object SongData. Berbeda dengan class biasa yang bisa diinstansiasi berulang kali, kata kunci object di Kotlin menerapkan design pattern Singleton. Ini menjamin bahwa di seluruh lifecycle aplikasi, memori perangkat hanya akan menyimpan satu buah salinan objek SongData ini saja. Pendekatan ini sangat efisien untuk menyediakan sumber data (*data source*) statis secara global.

Pada baris [4] sampai [69], variabel global val songs dideklarasikan yang diinisialisasi menggunakan fungsi listOf(...). Karena menggunakan listOf daripada mutableListOf, koleksi data yang dihasilkan bersifat *immutable* (tidak dapat dimodifikasi, ditambah, atau dihapus setelah aplikasi berjalan), memastikan data tetap aman dan konsisten saat dirender di lapisan UI, misalnya menggunakan RecyclerView. Di dalam blok listOf, dilakukan pemanggilan konstruktor Song(...) secara berulang untuk melakukan pemetaan dummy data. Setiap blok mengisi parameter sesuai kontrak data kelasnya secara berurutan, mengorganisasikan koleksi lagu.

**strings.xml (eng)**

Pada baris [1] sampai [17], elemen <resources> digunakan sebagai root tag yang secara mutlak mendefinisikan bahwa file ini berfungsi sebagai tempat penyimpanan value resource di dalam sistem Android.

Pada baris [2] sampai [8], dideklarasikan sekumpulan elemen <string> berukuran pendek. Elemen-elemen ini memetakan variabel ID spesifik seperti app_name, label_album, btn_listen dengan nilai antarmuka dalam bahasa Inggris.

Pada baris [10] sampai [16], dideklarasikan sekumpulan elemen <string> yang memuat paragraf panjang untuk mendeskripsikan informasi spesifik dari setiap lagu seperti desc_i, desc_flash, dan lainnya. Teks inilah yang direferensikan menggunakan ID, contohnya R.string.desc_i pada objek SongData.

**strings.xml (in)**

Pada baris [1] sampai [17], elemen <resources> kembali digunakan sebagai root tag. File ini secara khusus ditempatkan di dalam direktori values-id oleh sistem Android, terpisah dari strings.xml utama yang biasanya berada di folder values bawaan.

Pada baris [2] sampai [8], dideklarasikan elemen <string> untuk teks antarmuka berukuran pendek seperti tombol dan label yang telah diterjemahkan ke dalam Bahasa Indonesia. Atribut yang dibuat sama persis dengan yang ada di strings.xml versi bahasa Inggris.

Pada baris [10] sampai [16], dideklarasikan elemen <string> yang memuat paragraph terjemahan panjang untuk mendeskripsikan setiap lagu.

**item_song.xml**

Pada baris [2] sampai [10], komponen MaterialCardView digunakan sebagai root layout yang membungkus keseluruhan elemen kartu. Atribut android:layout_width="match_parent" memastikan kartu memenuhi lebar layar yang tersedia. Penggunaan atribut app:cardCornerRadius="16dp" memberikan efek membulat pada keempat sudut kartu, app:cardElevation="4dp" memberikan efek bayangan tiga dimensi, dan app:cardBackgroundColor="#FFC2D1" menetapkan warna latar belakang statis *Pink* Champagne secara eksplisit.

Pada baris [12] sampai [15], ConstraintLayout disarangkan di dalam MaterialCardView sebagai tata letak utama pengatur posisi komponen. Pendekatan ini efisien karena ConstraintLayout memungkinkan antarmuka yang kompleks dan datar

tanpa perlu menyarangkan banyak tata letak, sehingga performa rendering menjadi lebih optimal. Atribut android:padding="12dp" memberikan ruang kosong di sisi dalam agar elemen tidak menempel langsung pada tepi kartu.

Pada baris [17] sampai [24], komponen ShapeableImageView digunakan untuk menampilkan gambar cover album. Komponen diberi identitas @+id/img_song dengan dimensi statis 110dp x 110dp. Atribut android:scaleType="centerCrop" memastikan gambar memotong bagian tepinya agar proporsional dan tidak terdistorsi. Atribut app:shapeAppearanceOverlay="@style/RoundedCornerImage" adalah implementasi modern dari Material Design untuk memberikan modifikasi sudut membulat secara langsung pada gambar tanpa perlu membuat file drawable background terpisah.

Pada baris [26] sampai [36], TextView dengan ID tv_title bertugas merender teks judul lagu. Atribut android:layout_width="0dp" diterapkan. Posisi diikat di sebelah kanan gambar menggunakan layout_constraintStart_toEndOf dan di sebelah kiri teks tahun layout_constraintEnd_toStartOf, lebar 0dp akan memaksa teks judul untuk memanjang secara dinamis hanya pada sisa ruang kosong yang tersedia, mencegah teks bertumpuk dengan elemen lain jika judul terlalu panjang.

Pada baris [38] sampai [44], TextView dengan ID tv_year digunakan untuk mencetak tahun rilis. Komponen ini ada secara presisi di sudut kanan atas wadah menggunakan app:layout_constraintEnd_toEndOf="parent" dan app:layout_constraintTop_toTopOf="parent".

Pada baris [45] sampai [53], TextView dengan ID tv_label_album berfungsi sebagai label teks statis. Penggunaan android:text="@string/label_album" mendemonstrasikan pemisahan nilai teks dari file tata letak. Elemen ini diposisikan tepat di bawah judul lagu.

Pada baris [55] sampai [63], TextView dengan ID tv_album_name menampilkan nilai dinamis dari nama album. Sama seperti judul, elemen ini menggunakan android:layout_width="0dp" agar teks mengisi ruang kosong yang tersisa dari sebelah kanan label album hingga ke batas tepi kanan *parent*.

Pada baris [65] sampai [75], komponen Button dengan ID btn_link mempresentasikan tombol aksi Listen. Desain warnanya menggunakan teks merah muda (#FB6F92) dan latar belakang putih (@color/white). Secara tata letak, tombol ini tidak terikat pada *parent*, tapi menyelaraskan posisi atas dan bawahnya (constraintTop_toTopOf dan constraintBottom_toBottomOf) langsung terhadap tombol detail (btn_detail), sehingga keduanya selalu sejajar secara horizontal.

Pada baris [77] sampai [86], komponen Button dengan ID btn_detail digunakan untuk navigasi ke halaman spesifik. Tombol ini menggunakan warna kebalikan dari tombol Listen. Posisinya dikunci di sudut kanan bawah kartu (constraintBottom_toBottomOf="parent" dan constraintEnd_toEndOf="parent"s) dan berada di bawah teks nama album. Tombol ini bertindak sebagai jangkar vertical utama untuk tombol Listen yang ada di sebelahnya.

**fragment_home.xml**

Pada baris [2] sampai [7], komponen LinearLayout digunakan sebagai root layout yang membungkus seluruh halaman aplikasi. Penggunaan atribut android:orientation="vertical" menginstruksikan sistem untuk menyusun semua anak elemen di dalamnya secara lurus dari atas ke bawah. Atribut android:background="#FFE5EC" memberikan warna latar belakang merah muda pada keseluruhan layar.

Pada baris [9] sampai [28], komponen RelativeLayout digunakan sebagai wadah untuk bilah navigasi atas. Pemilihan ini memungkinkan elemen-elemen di dalamnya diposisikan relatif terhadap layar atau elemen lain, tanpa perlu menggunakan nested layout tambahan.

Pada baris [13] sampai [18], TextView menampilkan judul. Nilai teks dipanggil dari sumber daya terjemahan menggunakan @string/app_name, diceetak dengan ukuran 22sp, dan bold.

Pada baris [20] sampai [27], ImageButton dengan ID btn_language digunakan sebagai tombol interaktif untuk mengganti bahasa. Atribut

android:layout_alignParentEnd=”true” memaksa tombol untuk merapat secara absolut ke ujung kanan layar. Atribut app:tint=”#FB6F92” digunakan untuk mewarnai ulang ikon bawaan Android menjadi merah muda pekat.

Pada baris [30] sampai [37], komponen RecyclerView pertama dideklarasikan dengan ID rv_highlight. Komponen ini bertindak sama seperti LazyRow pada Jetpack Compose, yaitu untuk merender daftar lagu yang dapat digeser secara horizontal. Pemberian jarak tepi android:paddingStart=”8dp” dan android:paddingEnd=”8dp”, dipadukan dengan android:clipToPadding=”false”. Atribut clipToPadding=”false” memastikan bahwa saat pengguna menggeser daftar kartu lagu, kartu tersebut tetap dapat mengalir dan terlihat di area *padding* tanpa terpotong secara kasar, sehingga menciptakan carousel effect yang halus dan modern di pinggir layar.

Pada baris [39] sampai [43], komponen RecycleView kedua dideklarasikan dengan ID rv_songs. Ini bertindak sama seperti LazyColumn pada Compose, yang bertugas merender daftar lagu vertical. Atribut android:layout_width=”match_parent” dan android:layout_height=”match_parent” memastikan komponen ini mengisi seluruh sisa ruang layar yang ada di bawah *header* dan *carousel* horizontal, dengan diberikan sedikit jarak atas menggunakan android:layout_marginTop=”8dp”

**fragment_detail.xml**

Pada baris [2] sampai [6], komponen androidx.core.widget.NestedScrollView digunakan sebagai root layout. Karena panjang teks deskripsi lagu bisa bervariasi dan berpotensi melebihi tinggi layar perangkat, NestedScrollView memastikan seluruh konten di dalamnya dapat digulir ke bawah tanpa terpotong. Atribut android:background=”#FFE5EC” memberikan warna latar belakang solid merah muda muda pada keseluruhan layar.

Pada baris [8] sampai [12], sebuah LinearLayout disarangkan tepat di dalam NestedScrollView. Hal ini diwajibkan karena NestedScrollView hanya diizinkan memiliki satu komponen anak. Atribut android:orientation=”vertical” menginstruksikan sistem untuk menyusun semua elemen visual (gambar, teks, tombol)

secara lurus dari atas ke bawah. Atribut android:padding="16dp" memberikan jarak aman di keempat sisi layar agar konten tidak menempel langsung pada pinggiran perangkat.

Pada baris [14] sampai [20], komponen ShapeableImageView digunakan untuk menampilkan gambar cover album. Dimensinya ditetapkan secara statis sebesar 320dp x 320dp. Atribut android:layout_gravity="center" memastikan gambar berada tepat di tengah layar secara horizontal. Sama seperti pada rancangan kartu lagu, atribut android:scaleType="centerCrop" dan app:shapeAppearanceOverlay="@style/RoundedCornerImage" digunakan untuk menjaga proporsi gambar sekaligus memberikan modifikasi sudut membulat yang modern.

Pada baris [22] sampai [28], TextView dengan ID tv_detail_title bertugas merender teks judul lagu. Tipografi dibuat menonjol sebagai hierarki visual tertinggi dengan ukuran huruf ekstra besar (28sp), dicetak tebal (textStyle="bold"), dan diberikan jarak atas layout_marginTop="16dp" untuk memisahkannya dari kover album.

Pada baris [30] sampai [44], terdapat dua TextView berurutan, yaitu tv_detail_album dan tv_detail_year, yang menampilkan nama album dan tahun rilis. Keduanya menggunakan ukuran huruf standar 16sp dengan warna teks hitam pekat menggunakan @color/black dan jarak atas yang lebih rapat (4dp) untuk menandakan bahwa kedua informasi ini saling berkaitan erat.

Pada baris [46] sampai [53], TextView dengan ID tv_detail_desc dikhususkan untuk merender teks deskripsi atau informasi panjang terkait lagu. Pada komponen ini, disematkan atribut android:lineSpacingExtra="6dp". Menambahkan jarak spasi ekstra antarbaris teks secara signifikan meningkatkan tingkat readability pengguna saat membaca blok paragraf yang panjang di layar ponsel.

Pada baris [55] sampai [62], komponen Button dengan ID btn_back diletakkan di bagian paling bawah halaman sebagai aksi navigasi kembali. Atribut android:layout_width="match_parent" memaksa tombol jadi penuh mengisi lebar

layar yang tersedia. Desain tombol menggunakan warna latar merah muda pekat (#FB6F92) dengan teks berwarna putih, serta memanggil nilai teks dari sumber daya terjemahan melalui @string/btn_back.

**fragment_language.xml**

Pada baris [1] sampai [7], komponen LinearLayout digunakan sebagai *root layout* yang mengendalikan seluruh halaman. Atribut android:orientation="vertical" menginstruksikan agar elemen di dalamnya disusun lurus dari atas ke bawah. Terdapat penggunaan atribut android:gravity="center". Atribut ini memaksa seluruh elemen anak di dalam tata letak ini untuk tertarik dan berkumpul tepat di poros tengah layar, baik secara vertikal maupun horizontal. Hal ini memberikan estetika centered layout khusus untuk menu pengaturan. Atribut android:padding="24dp" memberikan ruang bernapas yang cukup luas di keempat sisi layar, sementara android:background="#FFE5EC" memberikan warna latar belakang merah muda.

Pada baris [9] sampai [15], TextView bertugas menampilkan teks judul halaman. Teks ini dipanggil secara dinamis menggunakan @string/setting_language untuk mendukung multi language. Tipografinya dibuat menonjol dengan ukuran 24sp dan dicetak tebal (textStyle="bold"). Atribut android:layout_marginBottom="32dp" disematkan untuk memberikan jarak spasi yang cukup renggang antara teks judul dengan deretan tombol di bawahnya.

Pada baris [17] sampai [23], komponen Button dengan ID btn_english dirender sebagai opsi untuk mengubah bahasa aplikasi ke bahasa Inggris. Atribut android:layout_width="match_parent" membuat tombol ini membentang penuh secara horizontal mengikuti batas *padding* 24dp dari *parent*. Desainnya diberikan warna latar belakang merah muda pekat (#FB6F92) melalui atribut backgroundTint dan warna teks putih murni (@android:color/white).

Pada baris [25] sampai [31], komponen Button kedua dideklarasikan dengan ID btn_indonesia sebagai opsi untuk Bahasa Indonesia. Desain, warna, dan proporsinya dibuat identik persis dengan tombol pertama untuk menjaga konsistensi antarmuka.

Karena pengaruh orientasi vertikal pada root layout, tombol ini secara otomatis akan tersusun tepat di bawah tombol bahasa Inggris tanpa memerlukan pengaturan posisi tambahan.

**activity_main.xml (XML)**

Pada baris [2] sampai [6], komponen androidx.constraintlayout.widget.ConstraintLayout digunakan sebagai root layout. Penggunaan ConstraintLayout memastikan fondasi tata letak yang kuat dan responsif. Atribut android:layout_width="match_parent" dan android:layout_height="match_parent" digunakan agar tata letak ini agar penuh mengikuti ukuran layar perangkat.

Pada baris [8] sampai [14], dideklarasikan komponen androidx.fragment.app.FragmentContainerView dengan ID nav_host_fragment. Komponen ini bertindak sebagai kanvas utama (*container*) bagi penerapan Navigation Component di Android berbasis XML (setara dengan NavHost pada Jetpack Compose).

Pada baris [10], atribut android:name="androidx.navigation.fragment.NavHostFragment" secara spesifik menginstruksikan sistem Android untuk menyuntikkan kelas NavHostFragment ke dalam wadah kosong ini. Pendekatan Single-Activity Architecture diterapkan, di mana aplikasi hanya memiliki satu Activity utama, sedangkan antarmuka visual (beranda, detail, pengaturan) diwujudkan dalam bentuk Fragment yang ditukar-tukar secara dinamis di dalam wadah ini.

Pada baris [13], atribut app:defaultNavHost="true" adalah pengaturan keamanan (fail-safe) yang vital. Pengaturan ini menghubungkan wadah navigasi dengan sistem navigasi fisik perangkat (system back button). Jika pengguna menekan tombol Back pada smartphone mereka, sistem akan secara otomatis memanggil layar sebelumnya dari pop back stack yang tersimpan di dalam container, bukan keluar dari aplikasi.

Pada baris [14], atribut app:navGraph="@navigation/nav_graph" bertugas menautkan wadah ini ke berkas peta rute navigasi (nav_graph.xml). Berkas tersebut berisi seluruh definisi rute layar beserta aksi perpindahannya, sehingga FragmentContainerView ini tahu persis fragmen mana yang harus dimuat pertama kali (start destination) dan bagaimana cara berpindah ke layar lainnya.

**SongAdapter.kt (XML)**

Pada baris [1] sampai [6], deklarasi package menunjukkan direktori file, diikuti oleh import untuk memanggil library fundamental pembentuk daftar antarmukan, yaitu RecyclerView, LayoutInflater, dan ViewGroup

Pada baris [8] sampai [12], dideklarasikan kelas SongAdapter yang mewarisi RecycleView.Adapter. Kelas ini adalah controller yang menghubungkan data logika dengan komponen visual RecyclerView. Pada konstruktor kelas ini, ada tiga parameter, list yang merupakan sumber data bertipe List<Song>, serta onDetailClick dan onLinkClick. onDetailClick dan onLinkClick menerapkan konsep Higher-Order Functions. Dengan menjadikan click event sebagai parameter fungsi, adapter tidak perlu tahu tentang logika navigasi atau pembukaan browser. Ini memastikan kode tetap mematuhi prinsip Single Responsibility, di mana adapter murni hanya mengurus data visual, logika aksi diserahkan ke Fragment atau Activity yang memanggilnya.

Pada baris [14], dideklarasikan kelas bersarang (inner class) ViewHolder. Kelas ini bertugas menyimpan cache referensi komponen UI dari satu kartu lagu. Karena menggunakan View Binding, konstruktor cukup menerima parameter binding, lalu meneruskan binding.root ke kelas induk RecyclerView.ViewHolder.

Pada baris [16] sampai [19], metode onCreateViewHolder(…) di-override. Sistem akan memanggil fungsi saat RecyclerView membutuhkan kartu visual baru karena kartu yang ada di memori tidak cukup untuk menutupi layar. Baris ItemSongBinding.inflate(…) bertugas menerjemahkan berkas item_song.xml menjadi objek visual menggunakan LayoutInflater, lalu mengembalikannya dalam bentuk objek ViewHolder yang baru.

Pada baris [21] sampai [36], metode onBindViewHolder(…) di-override. Di sini proses data binding terjadi. Fungsi ini dieksekusi setiap kali sebuah kartu bergulir masuk ke dalam layar. Variable song diekstrak dari koleksi list berdasarkan position saat itu. Dengan blok scope function holder.binding.appy {…}, proses penulisan menjadi lebih ringkas. Data dari objek song dimasukkan ke elemen UI secara presisi, imageResId disetel ke imgSong menggunakan setImageResource, sedangkan judul dan nama album dimasukkan ke tvTitle dan tvAlbumName. Di dalam blok ini, setOnClickListener didaftarkan pada tombol btnDetail dan btnLink. Saat tombol ditekan, tombol tersebut akan mengeksekusi parameter fungsi lambda (onDetailClick atau onLinkClick) yang telah dideklarasikan di awal kelas sambil membawa objek song yang relevan.

Pada baris [38] sampai [40], metode getItemCount() di-override untuk mengembalikan total Panjang data (list.size). Angka ini memberitahu RecyclerView berapa banyak total baris maksimum yang harus ia kelola untuk mengalkulasi proporsi scrollbar secara akurat.

**HomeFragment.kt (XML)**

Pada baris [1] sampai [13], deklarasi package menunjukkan direktori file, diikuti import library Android. Library yang dipanggil mencakup lifecycle antarmuka (LayoutInflater, ViewGroup, View), arsitektur Fragment, sistem navigasi modern (findNavController), pengelolaan tata letak RecyclerView (LinearLayoutManager, PagerSnapHelper), serta kelas FragmentHomeBinding yang dihasilkan secara otomatis oleh sistem View Binding.

Pada baris [15] sampai [17], dideklarasikan class HomeFragment yang mewarisi sifat dari kelas dasar Fragment. Di dalam kelas ini, terdapat implementasi pola backing property menggunakan _binding (yang bersifat nullable) dan binding (non-null). Berbeda dengan Activity, lifecycle antarmuka sebuah Fragment bisa dihancurkan dan dibuat ulang oleh sistem sementara objek kelas Fragment-nya sendiri masih hidup di memori. Penggunaan pola ini mencegah terjadinya memory leak dengan memastikan bahwa referensi ke elemen UI hanya diakses saat UI tersebut benar-benar ada.

Pada baris [19] sampai [26], metode onCreateView(…) di-override. Fase lifecycle ini bertanggung jawab untuk menciptakan dan menggambar (inflate) antarmuka visual. Berkas fragment_home.xml diterjemahkan menjadi objek Kotlin melalui FragmentHomeBinding.inflate(...), lalu akar tata letaknya (binding.root) dikembalikan ke sistem untuk ditampilkan ke layar.

Pada baris [28] sampai [53], metode onViewCreated(…) di-override. Fase ini dieksekusi tepat setelah antarmuka selesai digambar. Seluruh inisialisasi data dan logika interaksi didaftarkan.

Pada baris [31] sampai [40], variabel adapter diinisialisasi dengan membuat instansi dari SongAdapter. Data SongData.songs disuplak ke dalamnya beserta dua blok fungsi lambda untuk menangani aksi klik dari dalam kartu.

Pada baris [32] sampai [35], cara kerja logika onDetailClick adalah ketika kartu ditekan, sebuah Bundle dibuat untuk membungkus data lagu. Karena kelas Song sebelumnya telah mengimplementasikan antarmuka Serializable, objek song utuh dapat disisipkan ke dalam paket menggunakan putSerializabe("song", song). Selanjutnya, findNavController().navigate(…) dipanggil untuk mengekseksui perpindahan rute ke layar detail (action_home_to_detail) dengan membawa paket bundle tersebut.

Pada baris [36] sampai [39], logika onLinkClick menggunakan pendekatan implicit intent dengan aksi Intent.ACTION_VIEW yang menyertakan data URI hasil dari parsing url eksternal. Perintah startActivity(intent) kemudian akan memicu sistem operasi Android untuk membuka link tersebut di aplikasi pihak ketiga.

Pada baris [42] dan [43], pengelola tata letak untuk rvSongs (daftar vertikal) diatur menggunakan LinearLayoutManager(requireContext()) yang secara default mengurutkan elemen dari atas ke bawah. Setelah itu, adapter yang baru dibuat dihubungkan ke RecyclerView.

Pada baris [45] sampai [48], pengelola tata letak untuk rvHighlight (daftar horizontal) diatur dengan menyisipkan parameter LinearLayoutManager.HORIZONTAL. Lalu

ada penerapa kelas PagerSnapHelper() yang ditautkan ke rvHighlight. PagerSnapHelper memodifikasi perilaku scroll alami RecyclerView agar tidak berhenti sembarangan di tengah-tengah kartu, melainkan terkunci secara presisi ke satu kartu penuh setiap kali pengguna selesai menggeser layar, menduplikasi perilaku interaktif seperti fitur carousel atau ViewPager.

Pada baris [50] sampai [52], fungsi OnClickListener didaftarkan pada btnLanguage. Saat tombol ditekan, findNavController().navigate(...) akan membaca rute aksi action_home_to_language dari graf navigasi dan langsung mengganti layar saat ini dengan layar pengaturan bahasa.

**DetailFragment.kt (XML)**

Pada baris [1] sampai [9], deklarasi package menunjukkan direktori file, diikuti dengan sekumpulan import untuk memanggil library dasar Android. Pustaka ini mencakup penanganan lifecycle antarmuka (LayoutInflater, ViewGroup, View), arsitektur Fragment, pengelola navigasi (findNavController), serta kelas FragmentDetailBinding yang dihasilkan oleh sistem View Binding.

Pada baris [10] sampai [12], dideklarasikan class DetailFragment yang mewarisi kelas dasar Fragment. Sama halnya dengan HomeFragment, kelas ini menerapka pola backing property dengna variabel _binding dan binding. Implementasi ini sangat krusial di dalam arsitektur Fragment untuk memisahkan referensi antarmuka visual dari objek kelas itu sendiri, hal ini untuk menghindari memory leak.

Pada baris [14] sampai [21], metode onCreateView(…) di-override untuk menangani fase awal penciptaan visual. Di dalam blok ini, fragment_detail.xml diterjemahkan menjadi objek visual melalui metode FragmentDetailBinding.inflate(…), lalu akar tata letaknya (binding.root) dirender ke layar.

Pada baris [23] sampai [42], metode onViewCreated(…) di-override sebagai tempat utama untuk mengeksekusi logika setelah antarmuka selesai digambar.

Pada baris [26], ada proses penangkapan data. Argumen yang dikirimkan oleh HomeFragment sebelumnya melalui Bundle diekstrak menggunakan metode

getSerializable(“song”). Penggunaan operator as? (safe cast) di sini merupakan best practice Kotlin. Daripada melakukan force cast, as? akan secara aman mencoba mengonversi data menjadi tipe objek Song. Jika tipe datanya tidak cocok atau terjadi kesalahan, ia tidak akan membuat aplikasi crash, terhindar dari ClassCastException, melainkan hanya mengembalikan nilai null.

Pada baris [27] sampai [37], pengecekan null safety ( if(song != null) ) dilakukan sebelum memanipulasi antarmuka. Jika data lagu valid, data tersebut disuntikkan ke dalam komponen UI. Perintah getString(R.string.label_album) digunakan untuk memanggil teks statis dari resource (strings.xml), yang kemudian digabungkan dengan data dinamis menggunakan “$labelAlbum ${song.albumName}”. Pendekatan ini memastikan halaman detail tetap mendukung penuh fitur multi language yang diterapkan pada tingkat aplikasi.

Pada baris [39] sampai [41], fungsi setOnClickListerner disematkan pada tombol btnBack. Di dalamnya, metode findNavController().navigateUp() dieksekusi. Berbeda dengan navigate(), perintah navigateUp() secara spesifik menginstruksikan sistem navigasi untuk membuang halaman detail ini dari atas pop back stack dan mengembalikan pengguna ke halaman sebelumnya secara aman tanpa menciptakan instansi halaman baru.

Pada baris [44] sampai [47], metode onDestroyView di-override. Ini adalah bagian yang melengkapi pola backing property sebelumnya. Fase onDestroyView() dipanggil ketiga tampilan visual (view hierarchy) fragment dihancurkan oleh sistem, namun instansi kelas DetailFragment-nya mungkin masih ada di memori. Dengan secara eksplisit menetapkan _binding = null, maka memutus referensi ke komponen UI yang sudah mati tersebut. Hal ini memungkinkan Garbage Collector (pembersi memori Android) untuk segera membebaskan ruang memori, memastikan aplikasi berjalan optimal dan bebas dari memory leak.

**LanguageFragment.kt (XML)**

Pada baris [1] sampai [10], deklarasi package menunjukkan lokasi direktori file. Diikuti oleh sekumpulan import untuk memanggil pustaka arsitektur Android dasar (Bundle, LayoutInflater, View, ViewGroup, Fragment). Terdapat pemanggilan library androidx.appcompat.app.AppCompatDelegate dan androidx.core.os.LocaleListCompat. Kehadiran dua library ini menandakan bahwa Fragment ini mengimplementasikan fitur multi language modern dari Android API, yang memungkinkan aplikasi untuk mengubah bahasa antarmuka secara tanpa mengubah pengaturan bahasa sistem perangkat secara keseluruhan

Pada baris [11] sampai [13], dideklarasikan class LanguageFragment yang mewarisi sifat Fragment. Di dalamnya, terdapat lagi pola backing property melalui variabel _binding dan binding menggunakan FragmentLanguageBinding. Konsistensi penerapan pola ini di seluruh Fragment menunjukkan best practice dalam mencegah memory leak.

Pada baris [15] sampai [22], metode onCreateView(…) di-override. Fase lifecycle ini bertugas merender visual fragment_language.xml menjadi objek antarmuka yang nyata menggunakan metode FragmentLanguageBinding.inflate(…). Akar tata letaknya (binding.root) kemudian dikembalikan untuk digambar di layar.

Pada baris [24] sampai [33], metode onViewCreated(…) di-override untuk mendaftarkan logika interaksi pasca rendering visual. Fungsi onCheckListener disematkan pada dua tombol utama, yaitu btnEnglish dan btn Indonesia. Saat btnEnglish ditekan, fungsi akan mengeksekusi metode utilitas local setAppLocale(”en”). Sebaliknya, saat btnIndonesia ditekan, fungsi akan mengirimkan argument “id”.

Pada baris [35] sampai [38], dideklarasikan fungsi khusus private fun seetAppLocale(languageTag: String). Fungsi ini adalah otak dari sistem multi language dinamis di aplikasi ini. Pertama, perintah LocaleListCompat.forLanguageTags(languageTag) akan menangkap string kode bahasa (seperti “en” atau ”id”) dan mengonversinya menjadi objek daftar lokal yang

valid dan didukung oleh sistem Android. Kemudian, perintah AppCompatDelegate.setApplicationLocales(localeList) dieksekusi. Perintah ini menginstruksikan aplikasi untuk segera memperbarui konfigurasi bahasanya dan memuat ulang nilai teks dari direktori resource yang sesuai. Pembaruan ini terjadi secara instan tanpa memerlukan restart dari aplikasi.

Pada baris [40] sampai [43], metode onDestroyView() di-override. Sama dengan halaman beranda dan detail, fase ini menghancurkan referensi kelas ke komponen visual dengan menyetel _binding = null. Ini adalah langkah pembersihan mutlak agar Garbage Collector dapat membersihkan memori visual yang sudah tidak digunakan saat pengguna berpindah ke layar lain.

**MainActivity.kt (XML)**

Pada baris [1] sampai [5], deklarasi package menunjukkan direktori file. Diikuti import yang memanggil library dasar Android, yaitu Bundle untuk menyimpan status antarmuka, AppCompatActivity sebagai fondasi aktivitas Android modern, dan ActivityMainBinding

Pada baris [7], dideklarasikan class MainActivity yang mewarisi sifat dari AppCompatActivity. Berbeda dengan Jetpack Compose yang menggunakan ComponentActivity, pewarisan AppCompatActivity di sini bersifat mutlak. Hal ini diwajibkan karena aplikasi menggunakan arsitektur Navigation Component dengan NavHostFragment, di mana dukungan terhadap transaksi Fragment tingkat lanjut dan kompatibilitas sistem operasi Android versi lama (backward compatibility) hanya disediakan secara penuh oleh kelas ini.

Pada baris [8], dideklarasikan variabel binding menggunakan kata kunci private lateinit var. Atribut lateinit adalah mekanisme keamanan memori di Kotlin. Mengingat elemen antarmuka visual belum ada saat kelas pertama kali dibuat di memori, lateinit mengizinkan untuk mendeklarasikan variabelnya sekarang dan menunda pengisian nilainya hingga siklus pembuatan layar tiba, sehingga terhindar dari *null-safety checks* yang berlebihan di seluruh blok kode.

Pada baris [10] sampai [14], metode onCreate(…) di-override. Ini adalah lifecycle penting yang pertama kali dipanggil saat aplikasi diluncurkan.

Pada baris [11], perintah super.onCreate(savedInstanceState) dieksekusi untuk memastikan sistem Android menjalankan semua rutinitas persiapan bawaannya.

Pada baris [12], pada perintah binding = ActivityMainBinding.inflate(layoutInflater), alat *LayoutInflater* bertugas menerjemahkan atau merender kode XML mentah yang ada di activity_main.xml menjadi objek visual di dalam memori, lalu menyimpannya ke dalam variabel binding.

Pada baris [13], perintah setContentView(binding.root) dieksekusi. Jika pada arsitektur lama perintah ini diisi dengan pemanggilan referensi R.layout.activity_main, pada pendekatan modern ini, cukup menyuplai binding.root. Properti .root mewakili tata letak akar tertinggi dari berkas XML (dalam hal ini ConstraintLayout yang membungkus NavHostFragment). Perintah ini yang secara harfiah memproyeksikan seluruh kanvas antarmuka tersebut ke layar perangkat fisik. Mulai dari titik ini, MainActivity akan bertindak murni sebagai bingkai kosong (host) yang menampung pertukaran layar HomeFragment, DetailFragment, dan LanguageFragment.

# SOAL 2

**Soal Praktikum:**

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

## A. Pembahasan

Meskipun transisi paradigma pengembangan Android menuju Jetpack Compose dengan LazyColumn memberikan efisiensi yang sangat signifikan, terutama melalui reduksi kode *boilerplate* seperti hilangnya kebutuhan akan Adapter, ViewHolder, dan implementasi DiffUtil manual, arsitektur RecyclerView tetap dipertahankan dan diimplementasikan secara luas di industri. Hal ini didasari oleh beberapa faktor teknis dan arsitektural berikut:

1. Legacy Code dan Skala Proyek

   Sistem berbasis *View* tradisional (XML) masih menjadi fondasi bagi aplikasi *enterprise* skala besar yang memiliki jutaan baris kode. Melakukan *rewrite* total dari antarmuka berbasis RecyclerView ke LazyColumn tidak efisien secara komputasi, biaya, maupun waktu. Oleh karena itu, industri menerapkan pola migrasi bertahap. Modul lama yang stabil dipertahankan (seringkali melalui prinsip *interoperability*), sedangkan modul baru dikembangkan menggunakan Compose, sejalan dengan prinsip rekayasa perangkat lunak "*If it ain't broke, don't fix it.*"

2. Kontrol Mutlak pada Recycling dan Performa Mikro

   RecyclerView memberikan akses kontrol tingkat rendah untuk optimasi perangkat keras, yang penting bagi antarmuka yang sangat padat data.

   - RecycledViewPool, memungkinkan pembagian memori (*view pooling*) pada *nested lists*, sehingga menghasilkan efisiensi memori yang masif.

- DiffUtil Payloads, memungkinkan *rendering* ulang secara spesifik pada level komponen mikro, misalnya hanya memperbarui satu ikon interaksi tanpa memuat ulang seluruh struktur baris. Pada Jetpack Compose, proses *recomposition* berjalan otomatis dan berpotensi merender komponen secara berlebih jika *state management* tidak dirancang dengan presisi tinggi.

3. Kematangan Ekosistem dan Dependensi Eksternal

Ekosistem RecyclerView telah diekspansi selama lebih dari satu dekade. Mayoritas *library* pihak ketiga yang menangani integrasi kompleks seperti SDK periklanan atau *library* manajemen UI tingkat lanjut, masih dibangun di atas arsitektur RecyclerView.Adapter. Ketergantungan infrastruktur ini mengharuskan implementasi RecyclerView atau pembungkus AndroidView saat mengintegrasikan dependensi yang belum mendukung arsitektur deklaratif modern.

4. Fleksibilitas *Custom Layout* dan Animasi Tingkat Lanjut

Pemisahan logika tata letak melalui arsitektur LayoutManager pada RecyclerView memfasilitasi pembuatan tata letak geometris yang sangat spesifik dan non-standar. Selain itu, fitur seperti ItemAnimator dan ItemTouchHelper memberikan keandalan stabilitas yang sangat tinggi (*battle-tested*) untuk interaksi kompleks. Meskipun Compose terus berkembang mengejar kapabilitas ini, implementasi animasi struktural tingkat lanjut pada *view* tradisional memiliki referensi dan stabilitas historis yang lebih kuat.

# SOAL 3

**Soal Praktikum:**

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

## A. Pembahasan

Menurut pendapat saya, penerapan *Clean Architecture* sangat penting ketika membangun sebuah proyek Android menggunakan Kotlin karena arsitektur ini secara tegas memisahkan setiap komponen aplikasi berdasarkan tanggung jawabnya masing-masing.

Dengan membagi struktur proyek ke dalam lapisan-lapisan utama seperti *Presentation* (antarmuka), *Domain* (logika bisnis), dan *Data* (sumber data), kita bisa memastikan bahwa perubahan pada satu sisi, misalnya pembaruan desain UI tidak akan merusak sistem inti aplikasi. Selain itu, karena logika utama yang berada di lapisan *Domain* ditulis menggunakan Kotlin murni tanpa campur tangan *framework* Android, kode tersebut menjadi sangat mudah dan cepat untuk diuji (*Unit Testing*) secara terisolasi tanpa memerlukan emulator. Keuntungan terbesarnya adalah aplikasi kita menjadi mandiri dan tidak terikat pada teknologi atau *library* tertentu, sehingga jika suatu saat kita harus mengganti jenis basis data atau layanan jaringan, aturan bisnis aplikasi tetap utuh. Pada akhirnya, meskipun pada tahap awal terasa rumit dan mengharuskan kita menulis lebih banyak file, *Clean Architecture* membuat proyek lebih mudah dipelihara, diskalakan, dan dikerjakan secara kolaboratif oleh banyak pengembang tanpa menimbulkan konflik yang fatal.

## ADDITIONAL TASK

**Pembahasan Clean Code**

Membahas mengenai definisi *Clean Code* akan menghasilkan jawaban yang sangat beragam, karena setiap pengembang memiliki perspektif dan pengalaman unik yang

membentuk standar mereka sendiri. Untuk mendapatkan pemahaman yang lebih komprehensif, Robert C. Martin merangkum pandangan dari beberapa tokoh pionir dalam dunia rekayasa perangkat lunak mengenai apa itu kode yang bersih:

1) **Bjarne Stroustrup (Pencipta C++)**
   Ia menekankan pada aspek *elegance* dan efisiensi. Kode harus memiliki logika yang lugas agar *bugs* sulit bersembunyi, memiliki dependensi minimal untuk memudahkan pemeliharaan, serta penanganan kesalahan yang lengkap. Bagi Bjarne, kode yang bersih adalah kode yang hanya melakukan satu hal dengan sangat baik.

2) **Grady Booch (Penulis *Object Oriented Analysis and Design*)**
   Dari sudut pandang keterbacaan, Grady menyatakan bahwa kode yang bersih harus terbaca seperti prosa yang ditulis dengan baik. Kode tersebut tidak boleh mengaburkan niat desainer, melainkan harus penuh dengan abstraksi yang jelas (*crisp abstractions*) dan alur kontrol yang sederhana.

3) **"Big" Dave Thomas (Pendiri OTI)**
   Ia berpendapat bahwa kode yang bersih adalah kode yang dapat dibaca dan ditingkatkan oleh pengembang lain selain penulis aslinya. Lebih lanjut, Dave menegaskan bahwa kode tanpa *tests* adalah kode yang kotor, karena betapapun elegannya kode tersebut, tanpa pengujian maka ia menjadi tidak bersih.

4) **Michael Feathers (Penulis *Working Effectively with Legacy Code*)**
   Ia merangkum kualitas kode bersih ke dalam satu kata, yaitu kepedulian. Kode bersih selalu terlihat seperti ditulis oleh seseorang yang peduli dan menaruh perhatian besar pada detail agar tidak ada ruang untuk perbaikan yang terlihat jelas.

5) **Ron Jeffries (Penulis *Extreme Programming Installed*)**
   Ia menggunakan aturan kode sederhana milik Beck sebagai acuan, di mana kode bersih harus menjalankan semua tes, tidak mengandung duplikasi, menyatakan semua ide desain di dalam sistem, serta meminimalkan entitas

seperti kelas dan metode. Ron sangat berfokus pada pengurangan duplikasi dan peningkatan ekspresivitas kode.

6) **Ward Cunningham (Penemu Wiki)**

   Ia memberikan definisi yang berpusat pada ekspektasi pembaca, di mana kode bersih adalah kode yang setiap rutinnya terbaca persis seperti apa yang diduga. Kode tersebut disebut indah jika ia membuat bahasa pemrogramannya terlihat seolah-olah memang dibuat khusus untuk masalah yang sedang diselesaikan.

Penamaan adalah elemen yang ada di mana-mana dalam pembuatan perangkat lunak, mulai dari variabel, fungsi, argumen, hingga nama fail dan direktori. Mengingat intensitasnya yang begitu tinggi, Robert C. Martin menekankan pentingnya melakukan penamaan dengan baik melalui beberapa aturan praktis dalam Bab 2: *Meaningful Names*:

**Strategi Penamaan yang Efektif (*Meaningful Names*)**

1) **Use Intention-Revealing Names**

   Sebuah nama harus mampu menjawab alasan mengapa ia ada, apa yang dilakukannya, dan bagaimana ia digunakan. Jika suatu nama masih membutuhkan komentar untuk menjelaskan maksudnya, maka nama tersebut dianggap gagal mengungkapkan niatnya. Contohnya, mengganti variabel int d dengan elapsedTimeInDays akan jauh lebih informatif.

2) **Avoid Disinformation**

   Hindari meninggalkan petunjuk palsu yang dapat mengaburkan makna kode. Jangan menggunakan kata List jika tipe data sebenarnya bukan sebuah *list*, dan hindari penggunaan variabel yang secara visual mirip namun berbeda makna, seperti huruf l kecil dengan angka 1 atau huruf O besar dengan angka 0.

3) **Make Meaningful Distinctions**

   Jangan membedakan nama hanya untuk memuaskan *compiler*. Penggunaan rangkaian angka seperti a1, a2, ... aN bersifat tidak informatif. Begitu pula dengan penggunaan *noise words* seperti ProductInfo atau ProductData yang

tidak memberikan perbedaan makna yang jelas dibandingkan dengan kelas Product itu sendiri.

4) **Use Pronounceable Names**

Manusia sangat terbiasa dengan kata-kata, dan karena pemrograman adalah aktivitas sosial, nama-nama yang digunakan harus bisa diucapkan agar mudah didiskusikan tanpa terdengar konyol.

5) **Use Searchable Names**

Nama dengan satu huruf atau konstanta numerik sangat sulit untuk dilacak di seluruh korpus teks. Semakin luas *scope* suatu variabel, semakin panjang dan deskriptif pula nama yang harus diberikan agar mudah dicari.

6) **Avoid Encodings**

Di era modern dengan IDE yang canggih, menyandikan tipe data ke dalam nama (*Hungarian Notation*) atau memberikan awalan khusus pada variabel *member* (seperti m_) sudah tidak lagi diperlukan dan justru menjadi beban mental tambahan bagi pengembang.

7) **Avoid Mental Mapping**

Jangan memaksa pembaca kode untuk menerjemahkan nama variabel yang dibuat ke dalam konsep lain yang mereka pahami di kepala mereka, kejelasan adalah kunci utama seorang profesional.

8) **Class and Method Names**

Nama kelas dan objek harus berupa kata benda atau frasa kata benda seperti Customer atau Account. Sebaliknya, nama metode harus berupa kata kerja atau frasa kata kerja seperti postPayment atau save.

9) **Don't Be Cute**

Hindari penggunaan istilah lucu, bahasa gaul, atau lelucon budaya yang hanya dimengerti segelintir orang, pilihlah kejelasan di atas nilai hiburan.

10) **Pick One Word per Concept**

Konsistenlah menggunakan satu kata untuk satu konsep abstrak yang sama di seluruh basis kode. Menggunakan istilah fetch, retrieve, dan get secara bergantian untuk fungsi yang serupa hanya akan membingungkan pembaca.

**11) Don't Pun**

Hindari penggunaan kata yang sama untuk dua tujuan berbeda (*pun*). Jika metode add digunakan untuk menggabungkan dua nilai, jangan gunakan nama yang sama untuk memasukkan elemen ke dalam koleksi, gunakanlah insert atau append.

**12) Use Solution and Problem Domain Names**

Gunakan istilah teknis komputer (*solution domain*) seperti nama algoritma atau pola desain jika tersedia, karena pembaca adalah sesama pengembang. Jika tidak ada istilah teknis yang cocok, gunakan nama dari ranah bisnis atau masalah yang sedang diselesaikan (*problem domain*).

**13) Add Meaningful Context**

Berikan konteks yang jelas pada nama variabel dengan menempatkannya di dalam kelas, fungsi, atau ruang nama (*namespace*) yang tepat. Misalnya, mengelompokkan variabel alamat ke dalam kelas Address lebih baik daripada memberikan awalan addr pada setiap variabel.

**14) Don't Add Gratuitous Context**

Jangan menambahkan konteks secara berlebihan jika tidak diperlukan. Nama yang pendek biasanya lebih baik daripada yang panjang selama maknanya tetap jelas dan presisi.

## DAFTAR PUSTAKA

[1] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, Upper Saddle River, New Jersey: Prentice Hall, 2009.

# TAUTAN GIT

https://github.com/lisaryuna/Praktikum-Mobile